480

Control de versiones con BitBucket, SourceTree y flujo de trabajo Gitflow

Formación480

Índice

INTRO: ¿Por qué controlar versiones? Git y BitBucket.

Elementos del control de versiones.

Flujo de trabajo GitFlow.

- Nombrar ramas y mensajes de commit: conventional-commit.
- Pull request.
- La manera 480.

SourceTree.

END: Fuentes

Apéndice: Comandos git consola.

Formación de control de versiones con SourceTree y BitBucket con flujo de trabajo GitFlow

Jano Soler - 31 de mayo de 2021

¿Por qué controlar versiones?

Motivación y objetivos del control de versiones

Trabajo en equipo

Posibilita el trabajo concurrente por parte de varios miembros de un equipo, en cualquier archivo y en cualquier momento.

Backup

Bitbucket mantiene copias en servidores remotos y además cualquier repositorio local es un backup funcional.

Histórico

¿Qué se hizo?
¿Quién lo hizo?
¿Cuándo lo hizo?
¿Cómo lo hizo?

Restaurar versiones

Anticiparse al error en desarrollo de software: intenta no equivocarte, pero cuando ocurra, siempre podrás deshacerlo.

Git es un software de control de versiones diseñado por Linus Torvalds, para coordinar el proceso del mantenimiento de versiones de aplicaciones de cualquier envergadura. Su propósito es llevar registro de los cambios que realizan los usuarios en un repositorio distribuido, permitiendo, por tanto, coordinar el trabajo en grupo, haciéndolo además de manera muy eficiente y confiable.

+ Página oficial

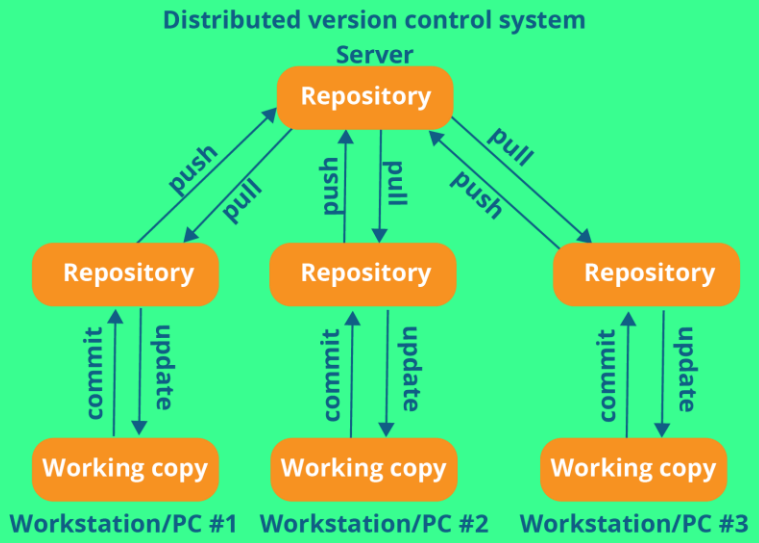
Bitbucket es una herramienta de alojamiento de código y colaboración basada en Git diseñada para equipos. Permite mediante una app consultar cualquier cambio que se haya producido en un repositorio, además de facilitar el control de los cambios (mediante la gestión de pull requests) e integra todo el control de versiones con herramientas de gestión de proyectos como Jira o Trello.

+ Página oficial

ATLASSIAN

Bitbuck

Elementos del control de versiones



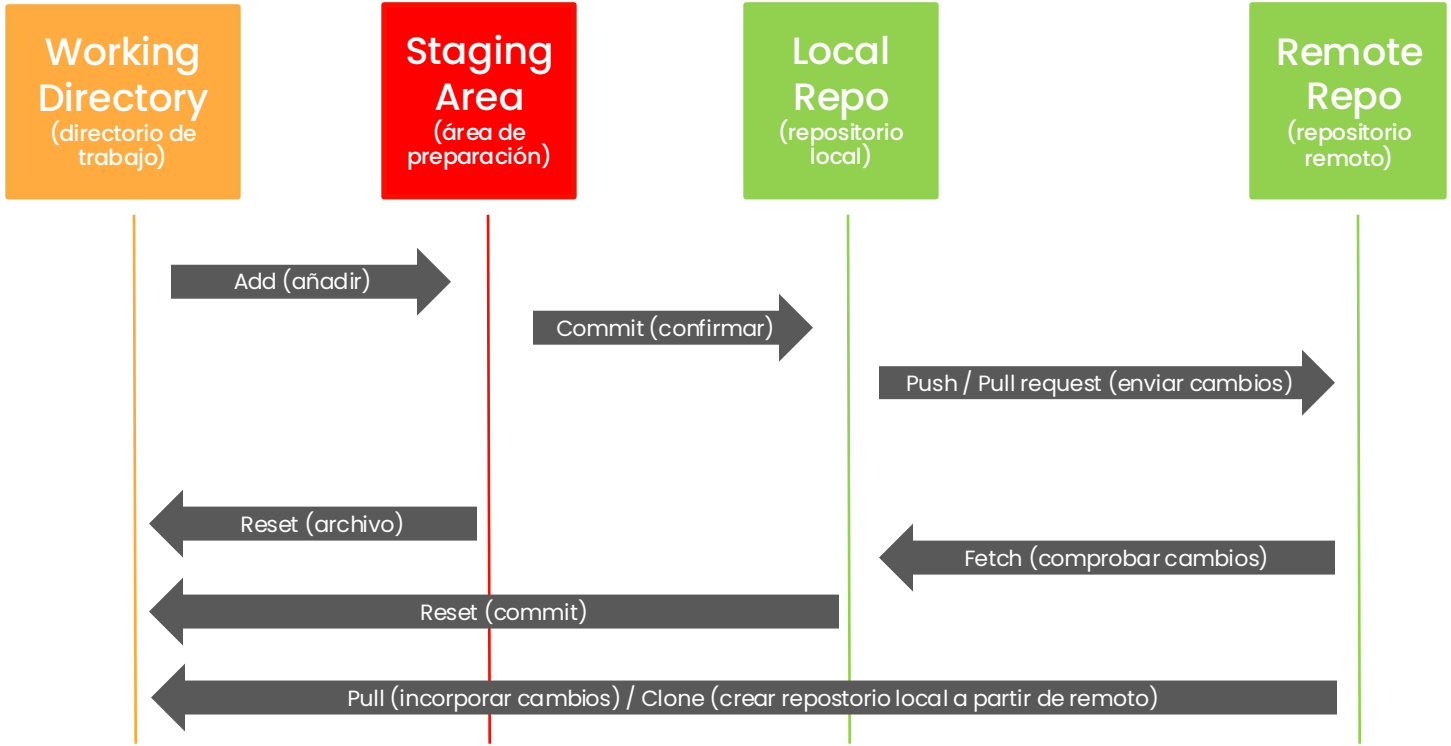
Cambios en el texto

Cuando hablamos de control de versiones… ¿de qué estamos hablando en realidad?

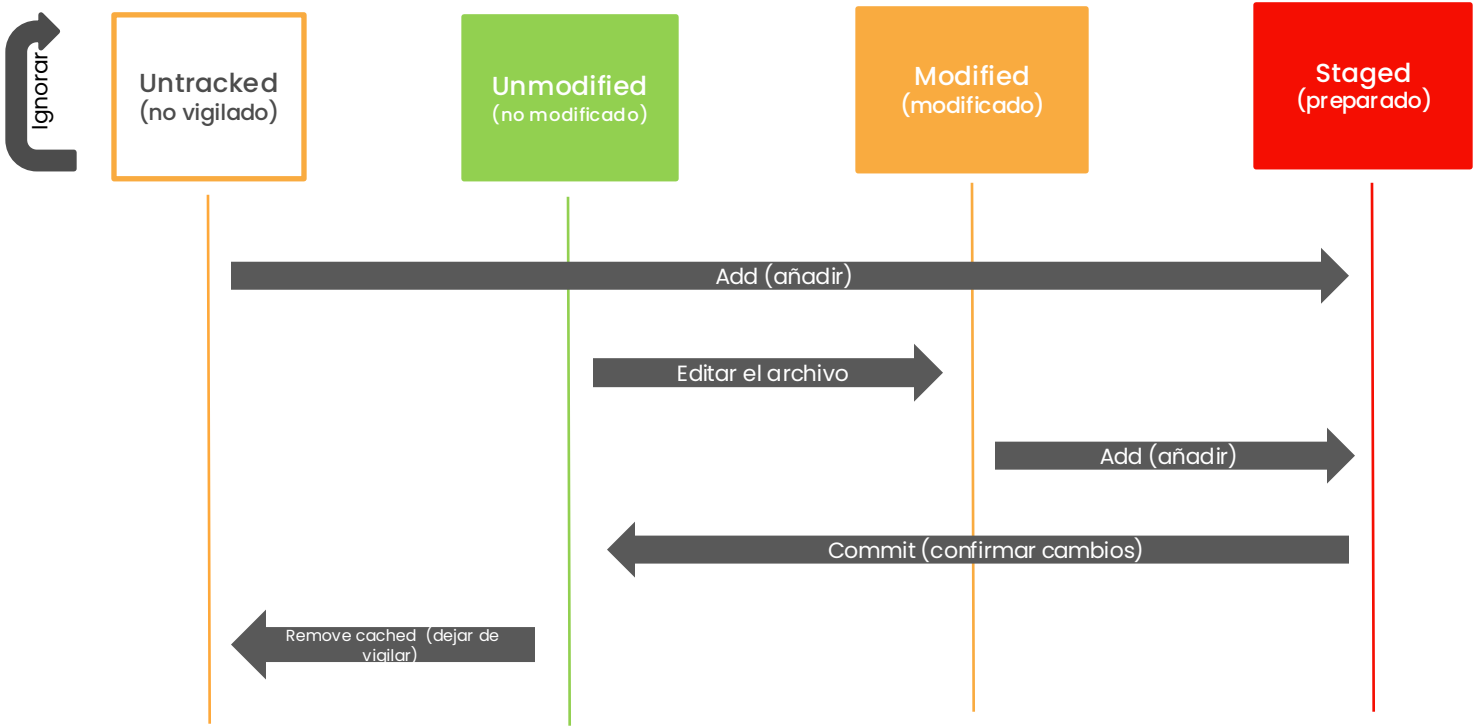
El concepto no es nada complejo: Git compara las diferencias en el texto de un conjunto de archivos, las lista y las muestra. A partir de ahí, disponemos de varias herramientas para crear una versión nueva incorporando esos cambios, de manera que quede registrada toda la actividad entre dos versiones diferentes, para poder recuperar otra versión y saber qué archivos se verían afectados por el cambio y qué contenido de esos archivos sería diferente.

Este proceso de mantenimiento y log de cambios que realiza git se realiza en varias áreas de trabajo diferentes.

Áreas de trabajo



Estados de un archivo

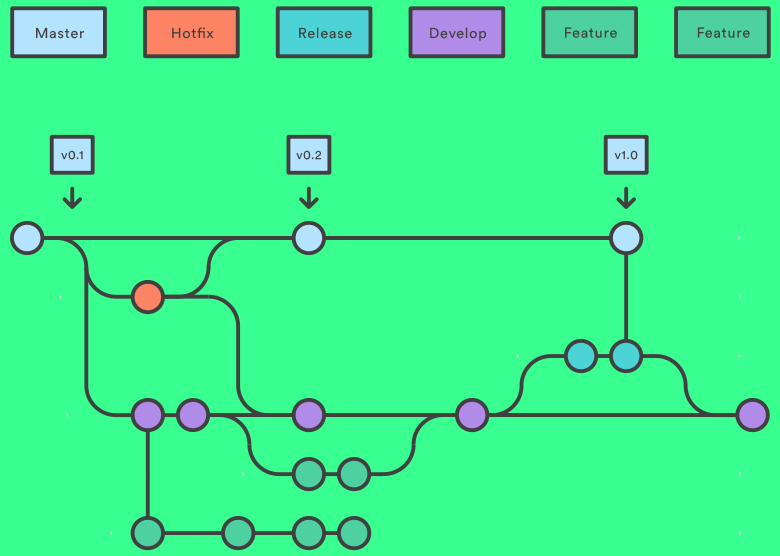


Creación y gestión de ramas (branching)

Al crear un repositorio de código con Git se crea una rama principal por defecto que se llama master. El flujo de trabajo habitual es utilizar esta rama para manterner el código validado en producción.

Para trabajar y añadir nuevas funcionalidades o reparar errores se crean nuevas ramas a partir de la rama master (o de otras). Una vez se crea el nuevo código y se valida, se incorpora a la rama master fusionando la rama donde se ha trabajado con el código en producción. Esta operación de fusión de ramas en Git se conoce como Merge.

GitFlow. La manera 480.



Flujo de trabajo en GitFlow

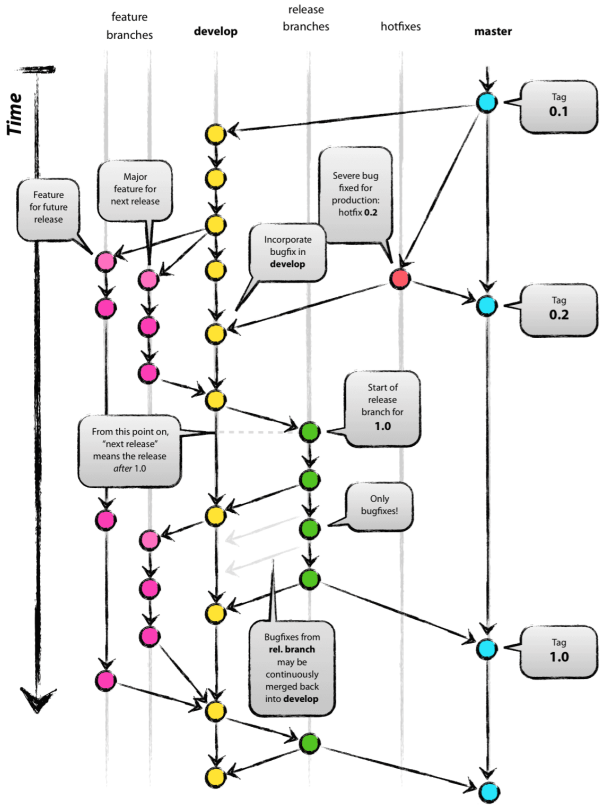
- **Master:** Rama que contiene el código en producción.

- **Develop:** Rama en la que añadimos cualquier modificación, previa a fusionar con Release (o, dependiendo del proyecto, directamente a master).

- **Feature:** Ramas individualizadas para cada nueva funcionalidad. (parten de develop y fusionan en develop).

- **Hotfix:** Ramas individualizadas para solucionar bugs (habitualmente parten de master y fusionan a master y develop).

- **Release:** Rama donde se incorpora develop periódicamente, paso a previo a fusionar con master. En esta rama solo se produce nuevo código para solucionar bugs (que no están en producción).



Nombres: Conventional-commit

Conventional-commit es una especificación para nombrar commits dentro de un repositorio. Usa una convención para facilitar la comprensión de los mensajes de los commits, tanto por los miembros del equipo como por máquinas.

En 480 nombramos los commit usando esta especificación, añadiendo al principio del mensaje, entre corchetes, la tarea a la que hace referencia en Jira, el gestor de proyectos.

A continuación, una de las etiquetas de conventional-commit, las más usadas serán:
-feat: commit asociado a una nueva funcionalidad.
-fix: commit asociado a solucionar un bug.

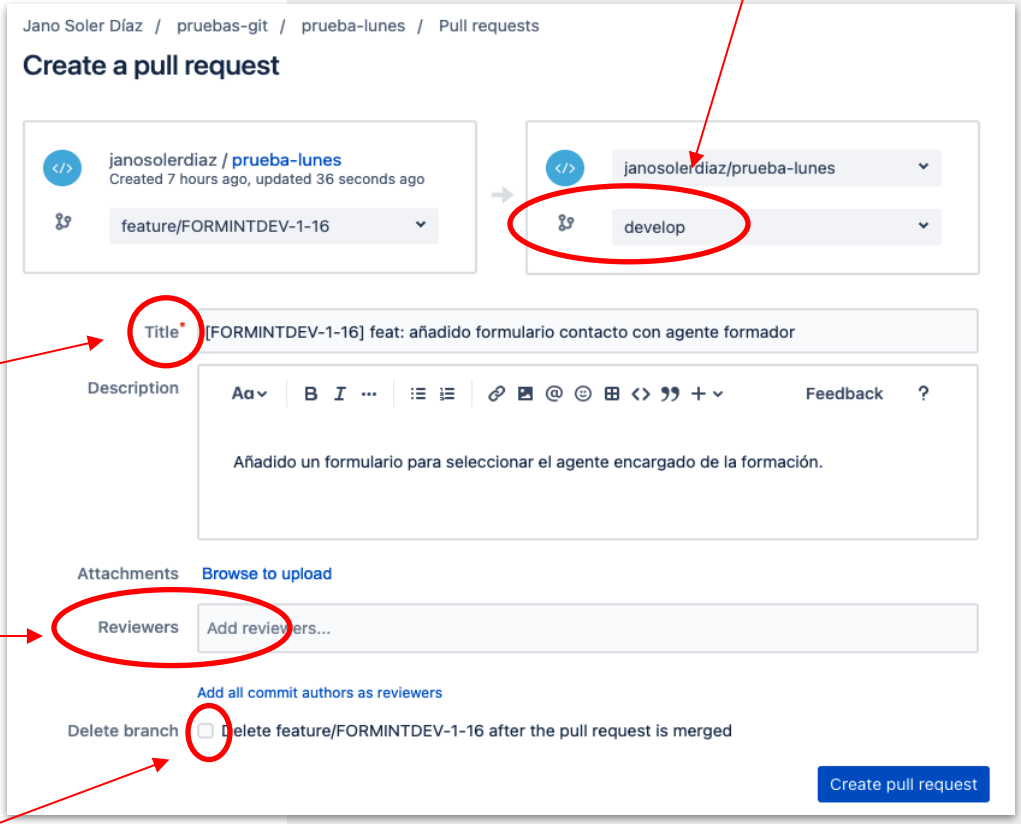
Por último, añadiríamos un texto breve y lo más descriptivo posible sobre el commit.

Ejemplo:
"[FORMINTDEV-1] feat: añadido formulario contacto en landing page"

Pull request

Proceso por el cual, desde BitBucket, se solicita la aprobación del código de un commit, paso previo a la fusión con la rama develop del repositorio remoto, que sería el último paso del flujo de trabajo.

El pull request siempre tendrá como objetivo la rama develop del remoto.



El título del pull request será, por defecto, el del último commit de la rama local.

Buscaremos y añadiremos como revisores a los usuarios de 480 según el proyecto y la tarea

Al final del proceso, una vez aprobado y se fusione con develop, la rama del feature debe eliminarse. Si seleccionas este check ese proceso será automático.

Gitflow en 480, ejemplo de flujo para nueva feature.

- Desde la rama develop de tu repositorio local creas una nueva rama para trabajar en la feature. Las nuevas ramas se nombran como el código de la tarea de Jira a la que enlazan y si trabajas desde línea de comandos debes crearla como "feature/codigo_tarea_en_Jira".

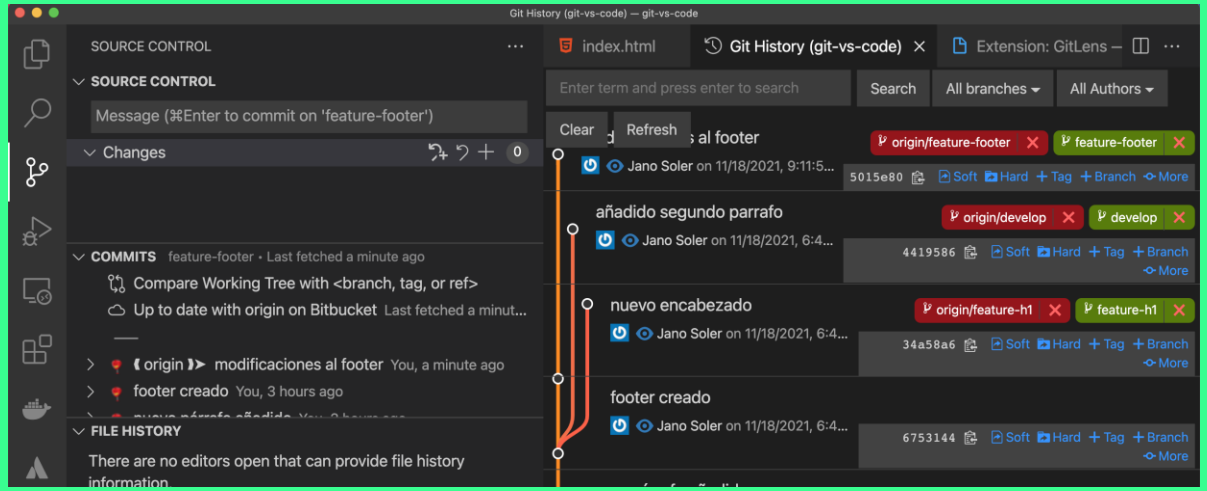
- Una vez concluido el trabajo haces commit (durante el trabajo puedes hacer todos los commits que quieras, evidentemente) y subes (push) a Bitbucket el trabajo realizado.

- Desde Bitbucket generas una pull request que irá siempre desde la rama de tu feature a la rama develop del remoto, solicitarás la revisión de los usuarios correspondientes y esperarás a que revisen y aprueben (o no) la pull request.

- En caso de que se apruebe deberás hacer merge con develop, pero no te dejará si hay conflictos. Estos conflictos tendrás que resolverlos y una vez este proceso se haya acabado ya te dejará fusionar la rama.

- En el caso de que no se apruebe el pull request deberás revisar tu código para modificarlo según las sugerencias de los revisores y volver a solicitar la aprobación.

VS Code.


Git History (git-vs-code) — git-vs-code
SOURCE CONTROL
SOURCE CONTROL
Message (⌘Enter to commit on 'feature-footer')
Changes
0
COMMITS feature-footer • Last fetched a minute ago
Compare Working Tree with <branch, tag, or ref>
Up to date with origin on Bitbucket Last fetched a minut...
origin modificaciones al footer You, a minute ago
footer creado You, 3 hours ago
FILE HISTORY
There are no editors open that can provide file history information
index.html
Git History (git-vs-code)
Extension: GitLens
Enter term and press enter to search
Search
All branches
All Authors
Clear
Refresh
al footer
origin/feature-footer
feature-footer
Jano Soler on 11/18/2021, 9:11:5...
5015e80
Soft Hard Tag Branch More
añadido segundo parrafo
origin/develop
develop
Jano Soler on 11/18/2021, 6:4...
4419586
Soft Hard Tag Branch More
nuevo encabezado
origin/feature-h1
feature-h1
Jano Soler on 11/18/2021, 6:4...
34a58a6
Soft Hard Tag Branch More
footer creado
Jano Soler on 11/18/2021, 6:4...
6753144
Soft Hard Tag Branch More

Cambios a un archivo.

index.html — git-vs-code

SOURCE CON...

Message (⌘Enter to commit on 'd...

Changes 1

index.html M

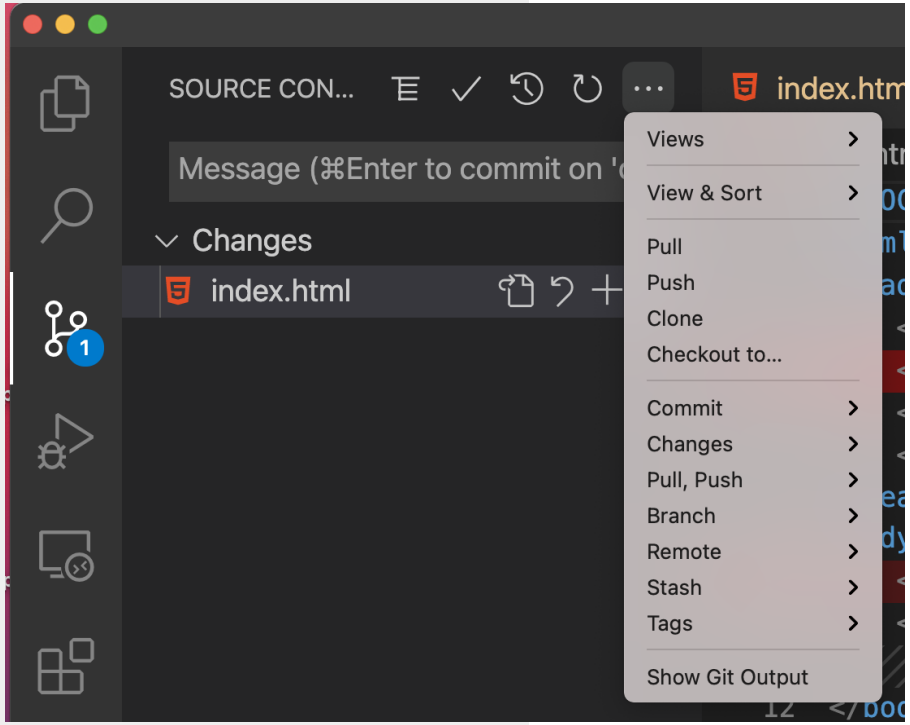
index.html M | index.html (Working Tree) M | Git History (git-v

index.html > html > body

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5—     <meta http-equiv="X-UA-Compatible"
6      <meta name="viewport" content="wi
7      <title>Probando git</title>
8  </head>
9  <body>
10—    <p>Probando git con visual studio
11     <p>Un segundo párrafo</p>
12 </body>
13 </html>
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="wic
6      <title>Probando git</title>
7  </head>
8  <body>
9+     <p>Git con visual studio code</p>
10     <p>Un segundo párrafo</p>
11+    <p>Un tercer párrafo</p>
12 </body>
13 </html>
```

Comandos git.



Histórico.

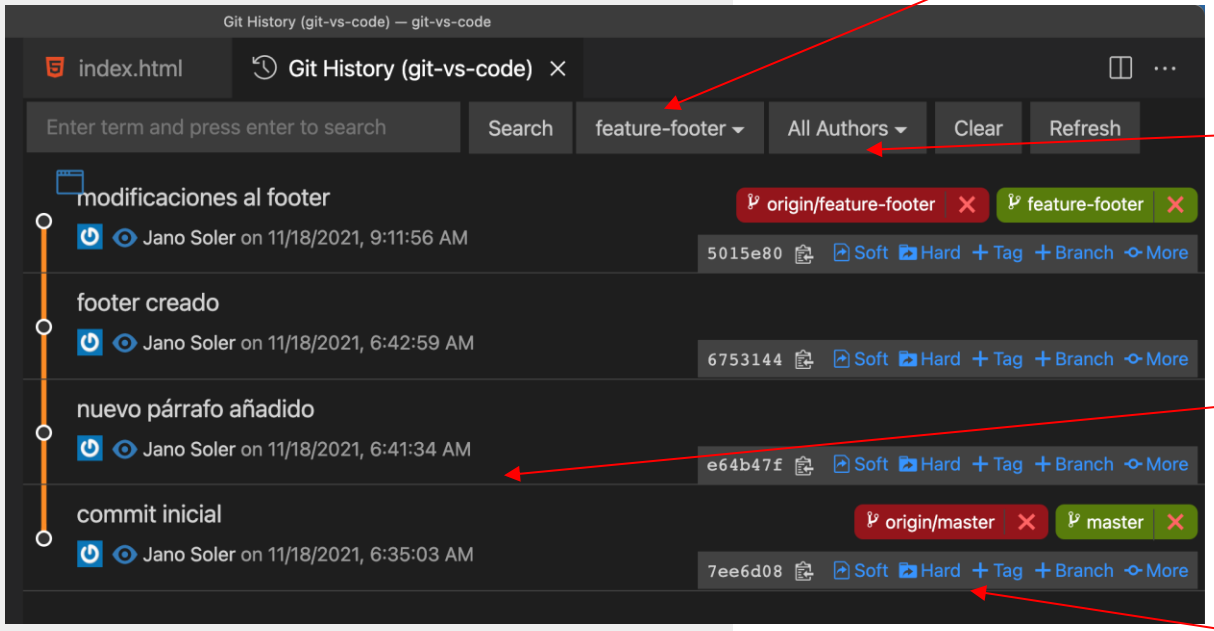
Hist´órico de commits por ramas

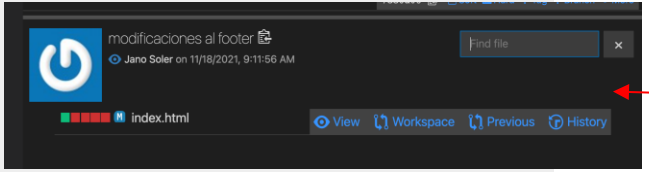
Filtrar commits por autor

Timeline de commits

Hash code, posibilidad de hacer hard y soft reset y de tagear el commit

Info del commit individual, con posibilidad de observar los cambios con el archivo actual o con el commit anterior





Git lens plugin.

index.html — git-vs-code

SOURCE CONTROL

Message (⌘Enter to commit on 'd...

Changes 1

index.html M

COMMITS develop • Last fetched 18 minu...

Compare Working Tree with ...

Up to date with origin on Bitb...

origin añadido segundo ...

nuevo párrafo añadido You, 3...

FILE HISTORY index.html

Uncommitted changes You,... M

origin añadido segund... M

nuevo párrafo añadido You, ... M

master, ... commit inici... A

BRANCHES (4)

feature-footer origin/featur...

develop origin/develo... ✔

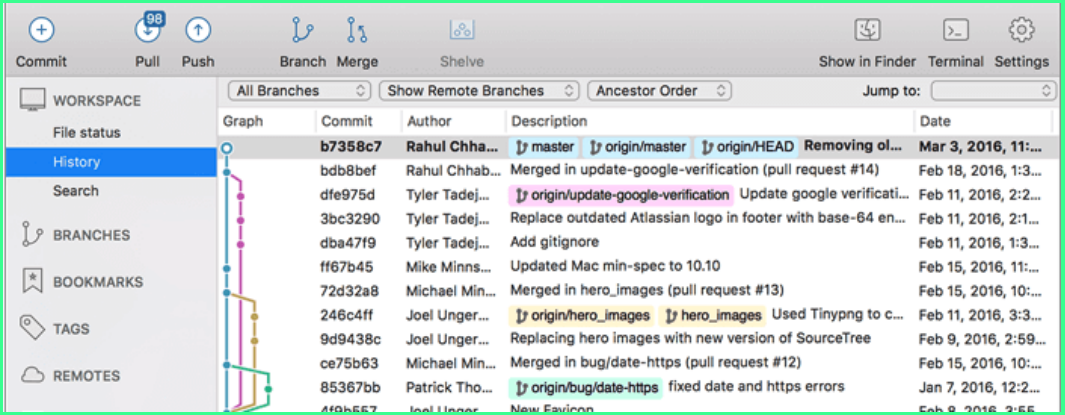
index.html M | index.html (Working Tree) M

index.html > html > body

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5-     <meta http-equiv="X-UA-Compatible"
6      <meta name="viewport" content="wi
7      <title>Probando git</title>
8  </head>
9  <body>
10-    <p>Probando git con visual studio
11     <p>Un segundo párrafo</p>
12 </body>
13 </html>
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="w
6      <title>Probando git</title>
7  </head>
8  <body>
9+     <p>Git con visual studio code</p
10     <p>Un segundo párrafo</p>
11+    <p>Un tercer párrafo</p>
12 </body>      You, 3 hours ago • comm
13 </html>
```

SourceTree.


Commit
98
Pull
Push
Branch
Merge
Shelve
Show in Finder
Terminal
Settings
WORKSPACE
File status
History
Search
BRANCHES
BOOKMARKS
TAGS
REMOTES
All Branches
Show Remote Branches
Ancestor Order
Jump to:
Graph
Commit
Author
Description
Date
b7358c7 Rahul Chha... master origin/master origin/HEAD Removing ol... Mar 3, 2016, 11:...
bdb8bef Rahul Chhab... Merged in update-google-verification (pull request #14) Feb 18, 2016, 1:3...
dfe975d Tyler Tadej... origin/update-google-verification Update google verificati... Feb 11, 2016, 2:2...
3bc3290 Tyler Tadej... Replace outdated Atlassian logo in footer with base-64 en... Feb 11, 2016, 2:1...
dba47f9 Tyler Tadej... Add gitignore Feb 11, 2016, 1:3...
ff67b45 Mike Minns... Updated Mac min-spec to 10.10 Feb 15, 2016, 11:...
72d32a8 Michael Min... Merged in hero_images (pull request #13) Feb 15, 2016, 10:...
246c4ff Joel Unger... origin/hero_images hero_images Used Tinypng to c... Feb 11, 2016, 3:3...
9d9438c Joel Unger... Replacing hero images with new version of SourceTree Feb 9, 2016, 2:59...
ce75b63 Michael Min... Merged in bug/date-https (pull request #12) Feb 15, 2016, 10:...
85367bb Patrick Tho... origin/bug/date-https fixed date and https errors Jan 7, 2016, 12:2...

Sourcetree es una herramienta que enmascara todos los comandos de Git en una interfaz gráfica, haciendo más amigable y visual el proceso del control de versiones, pudiendo desde la misma interfaz ver el historial de commits y los cambios realizados en el código. Tiene funcionalidad para facilitar el trabajo con la metodología Git Flow.

+ Página oficial

Sourcetr

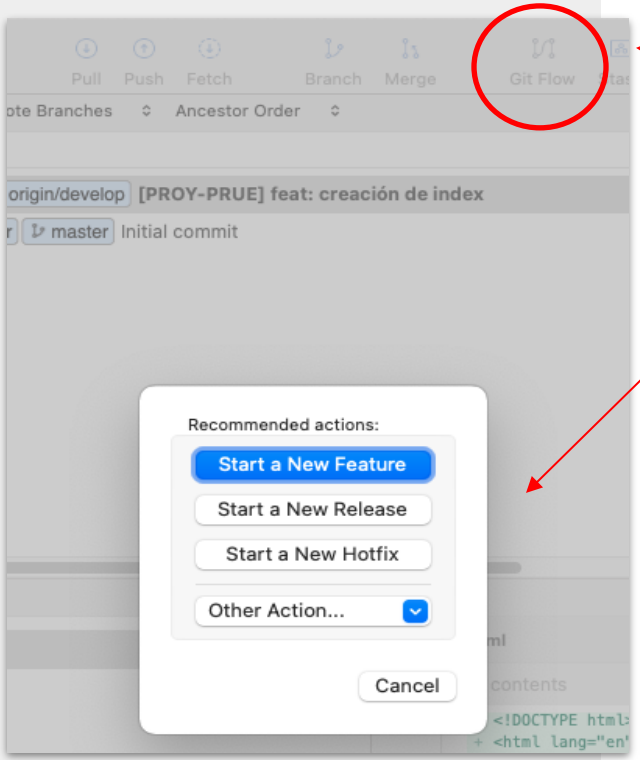
Acciones disponibles (comandos git)

Zona con el histórico de commits por ramas. Puedes filtrar una sola rama, por ramas locales o remotas o tener el histórico completo de todas las ramas

Zona con los archivos que tienen cambios (o se han añadido) según el commit seleccionado en el área del histórico.

Zona que muestra las modificaciones del código del archivo seleccionado en la zona de archivos del commit.

Información del commit seleccionado en el área del histórico.



Cuando seleccionamos la herramienta de Git Flow por primera vez desde un repositorio, crea todas las ramas necesarias para trabajar con este flujo.

Cuando lo seleccionamos en posteriores ocasiones aparecerá un menú que nos permite automatizar el proceso de creación de una nueva rama de tipo feature, hotfix o release.

END

Fuentes:

- Tutoriales git y bitbucket de Atlassian: https://www.atlassian.com/es/git/tutorials
- Git FLow (artículo original): https://nvie.com/posts/a-successful-git-branching-model/
- Conventional-commits: https://www.conventionalcommits.org/en/v1.0.0/

Agradecer la colaboración y supervisión de Sabin Petru.

Apéndice: comandos básicos terminal Git

Comandos básicos

- `git init` : inicializa un nuevo repositorio en el directorio actual. Sólo se debe ejecutar una vez en la vida del respositorio.

- `git status`: nos devuelve información relacionada con los archivos modificados distinguiendo entre los añadidos al staging area y los untracked.

- `git add ["nombre de archivo" | . ]` : añade el archivo especificado con el nombre al staging area o todos los archivos modificados o no rastreados si en lugar del nombre del archivo añadimos un punto (.)

- `git rm ["nombre de archivo" | . ]` : elimina del staging area los archivos (o el archivo si especificamos un nombre en lugar de todos) pendientes de confirmación.

- `git commit -m "mensaje"`: confirma los cambios en el staging area.

- `git log`: muestra información relativa a todos los commits del repositorio. Cada commit tiene un código asociado para identificarlos unívocamente.

-`git reset --hard IDENTIFICADOR`: elimina todos los commits a partir del código de commit que escribamos en IDENTIFICADOR.

Comandos básicos

- `git clone "url"`: descargar el proyecto alojado en la url a nuestro repositorio local. Al clonar se crea un enlace desde el repositorio local con el remoto.

- `git remote add origin "url"`: enlazar un repositorio local con uno remoto, dándole, en este caso, el nombre origin (podrían existir más de un remoto enlazado).

- `git pull`: descargarse los cambios de un remoto enlazado.

- `git push origin master`: mandar cambios del local al remoto.

- `git checkout "nombre_de_la_rama"`: cambiar de rama a la especificada en nombre.

- `git checkout -b "nombre_de_la_nueva_rama"`: crea una nueva rama con el nombre especificado y cambia a la rama.

- `git merge "nombre_de_la_rama_a_fusionar"`: fusiona la rama especificada con el nombre con la rama en la que nos encontremos situados en el momento de ejecución del comando.

- `git branch -d "nombre_de_la_rama"`: elimina la rama especificada.

480

BME Growth

www.cuatroochenta.com